Technical Report: Weather Service API Development

1. Document Link

(1) Github:

https://github.com/WENGLINGDIDI/Web-Services-and-Web-Data.git

(2) Pythonanywhere:

http://xiaoshutiao.pythonanywhere.com

(3) Postman Online API document:

https://documenter.getpostman.com/view/42982859/2sBXqDu4Qo

2. Technology Stack & Justification

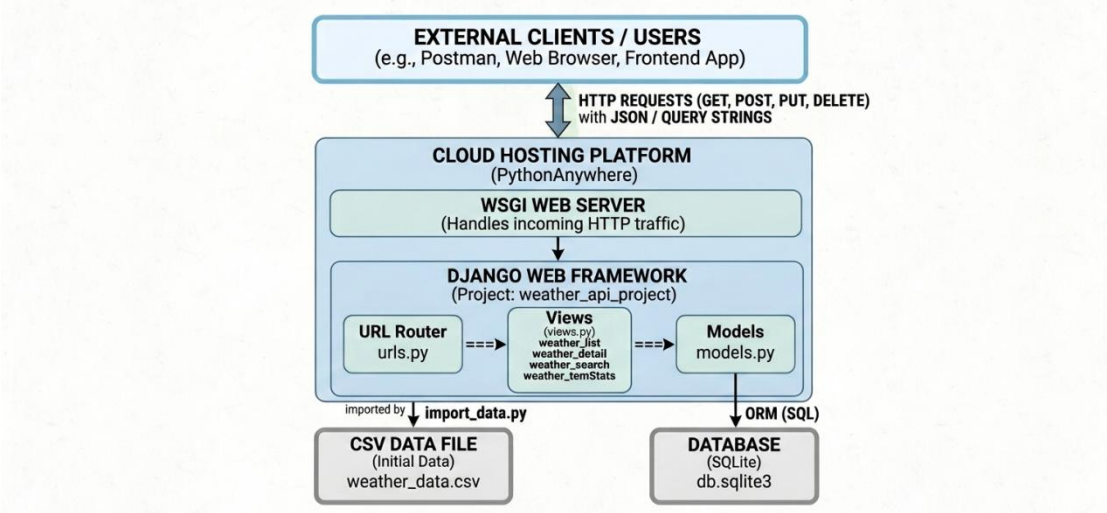
(1) Deep Evaluation of Programming Language and Core Framework:

For this project, I chose Python as the main programming language and Django as the core web framework. The reason for choosing Python is not only its concise syntax and excellent readability, but also its unparalleled ecosystem in the field of data science and data processing. This ensures the scalability of the weather service in future integration with machine learning prediction algorithms. As for the framework, I chose Django. This decision was based on Django's "out-of-the-box" concept. Its powerful object-relational mapping system greatly simplifies the logic of database interaction. In this project, I used Django ORM to efficiently manage the WeatherRecord model, avoiding the complexity and security risks (such as SQL injection) brought by writing raw SQL queries. Additionally, Django's built-in management interface provides a powerful tool for initial data auditing and manual maintenance.

(2) Database Selection and Cloud Deployment Strategy: This database uses the SQLite system. It fully complies with the standard SQL specifications and eliminates the environmental overhead required by heavyweight databases like PostgreSQL or MySQL. In terms of deployment, I chose the PythonAnywhere cloud platform. By

configuring the WSGI interface and the Python virtual environment, I successfully migrated the local development environment to the cloud. This process not only enhanced my proficiency in Linux server configuration, but also enabled me to gain a deeper understanding of the key differences between the development environment and the production environment, particularly in terms of security, static file services, and cross-origin resource sharing.

3. Architectural Design and Functional Implementation



(1) Advanced Practice of RESTful Architectural Style: This system strictly adheres to the RESTful API design principles and achieves its state transitions through standard HTTP methods. I have designed an intuitive URL structure, such as using /api/weather/ to represent a collection resource and using /api/weather/<int:pk>/ to locate a specific record. In terms of functionality implementation, I have adopted more than just basic CRUD operations to meet the requirements of practical application scenarios. For example, to address the query pressure that may arise from large datasets, I developed a dedicated weather_search endpoint. This endpoint supports multi-dimensional filtering through query strings. By leveraging Django's "chain query" feature, I have achieved flexible combinations of filtering conditions

such as conditions, minimum temperature, and maximum temperature, significantly enhancing the flexibility of the interface and reducing unnecessary data transmission.

(2) **Data Analytics and Processing Logic:** To enhance the practicality of the API, I implemented an analysis endpoint. This feature utilizes Django's aggregation functions to directly calculate the average temperature, maximum/minimum temperature, and average humidity at the database level. This "close to the data" design concept avoids the performance overhead caused by loading a large amount of raw data into memory for Python-side processing, demonstrating the commitment to optimizing the backend performance. Additionally, to ensure the integrity of the initial dataset, I developed an automated script named "import_data.py". This script parses the original "weather_data.csv" file, handles missing values, and performs data type conversion to ensure that the database is filled with high-quality baseline data from the very beginning.

4. Testing, Challenges, and Lessons Learned

(1) **Comprehensive Testing Lifecycle via Postman:** During the testing phase, I used Postman to build a complete lifecycle from unit tests to integration tests. I created dedicated test sets for each endpoint, covering a wide range of boundary cases. During the testing process, I paid great attention to handling "exceptional scenarios". For instance, I tested how the system would respond when a user sent an incomplete JSON data body in a POST request or entered non-numeric characters for the temperature parameter in the search endpoint. By using try...except blocks to catch ValueError and DoesNotExist exceptions, I ensured that the system returned the appropriate 400 error request or 404 not found status code.

(2) **Challenges and Solutions:** The biggest challenge lies in the

deployment configuration on PythonAnywhere. Initially, the code running locally failed to run on the cloud due to the "module not found error". By troubleshooting, I realized that this was caused by the virtual environment not being activated and the mismatch of dependencies. By setting an automatic activation script in the.bashrc file and strictly maintaining requirements.txt through pip freeze, I successfully synchronized the environment. Another challenge was the semantic use of HTTP status codes. For example, in the DELETE function, I initially returned 200 OK. But after studying the RESTful standard, I updated it to 204 No Content to more accurately convey the information that the operation was successful and no response body was needed.

5. Limitations and Future Development

(1) **Analysis of Current System Limitations:** Although the current system is functionally complete, there is still room for improvement in terms of security. This API is currently fully public and lacks authentication and authorization mechanisms. Any user with the URL can perform write operations, which is unacceptable in a production environment. Additionally, the statistical function is relatively fixed. Although it can calculate the average value, it currently cannot perform time series trend prediction. Data storage relies on the local SQLite file, and under extremely high concurrency, it may encounter a write lock bottleneck problem.

(2) **Vision for Future Expansion and Academic Integration:** In future iterations, I plan to introduce an authentication system based on JSON Web Tokens (JWT) and refactor the existing logic using the Django REST Framework.

6. Generative AI Declaration

(1) **Scope and Method of AI Usage:** During the development of this project, I used Generative AI (Gemini) as a strategic auxiliary tool. AI was primarily utilized for: Comparing the pros and cons of Django vs. Flask and optimizing the hierarchical structure of URL routing.Generating boilerplate code for view functions and troubleshooting path mapping errors for Django templates and static files. Drafting the initial versions of the README.md and the structural outline of this technical report. Also, some description pictures are generated by the AI.

(2) **Critical Analysis of AI Integration:** AI has played a crucial role as an "advanced pair programmer", significantly accelerating the development cycle. However, AI-generated content must undergo manual review. For instance, when dealing with complex ORM queries, AI sometimes proposes deprecated code or logic that does not match my specific data model. I will conduct manual unit tests to correct and refactor these situations. I believe that the responsible use of AI not only shortens the development time but also enables developers to focus on higher-level architectural design. All core business logic, testing procedures, and documentation optimization are led and completed by me, while AI merely functions as an advanced productivity auxiliary tool.